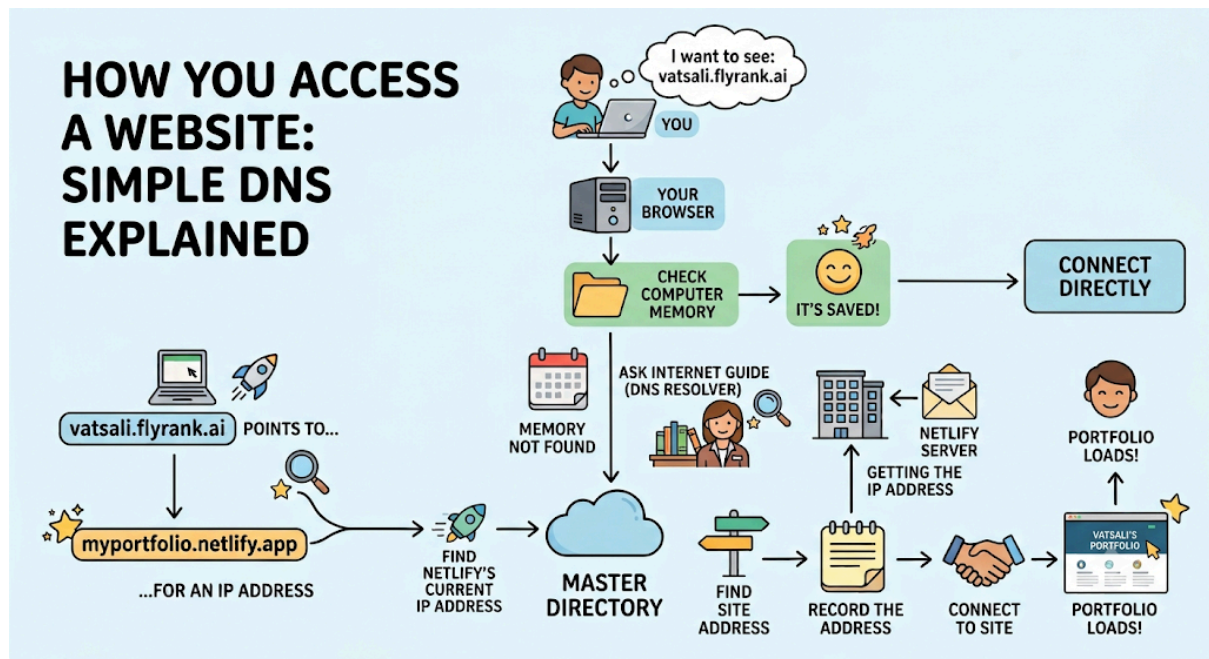


Tried to create a flowchart of how to explain it to a non tech guy, made a flowchart usign gemini and my knowledge



DNS Walkthrough: Connecting My FlyRank Subdomain to My Portfolio

Objective

My portfolio is currently deployed on Netlify using its free domain. Once FlyRank provides me with a custom subdomain (for example, `vatsal.flyrank.ai`), I will connect that subdomain to my existing Netlify deployment using DNS.

The goal is to make both URLs serve the same website without redeploying or changing the application's code.

Current Setup

At present, my portfolio is hosted on Netlify.

`https://myportfolio.netlify.app`

The website is already publicly accessible over HTTPS.

Future Setup

After receiving a FlyRank subdomain, the website should be accessible from:

`https://urvi.flyrank.ai`

Both URLs should point to the same website.

Understanding DNS

Every device connected to the Internet has an IP address, but IP addresses are difficult for humans to remember.

Instead, we use domain names such as:

`google.com`

`github.com`

`vatsal.flyrank.ai`

The Domain Name System (DNS) is responsible for translating these human readable names into IP addresses that computers can use.

How DNS Resolution Works

When a user types:

`https://urvi.flyrank.ai`

the browser does **not** immediately know where the website is hosted.

Instead, it:

1. Checks whether it already has the answer cached.
2. If the cached record has expired or doesn't exist, it asks a DNS Resolver.
3. The DNS Resolver contacts the authoritative DNS server.
4. DNS returns information that eventually leads to the server's IP address.
5. The browser connects to that server and downloads the website.

This means the browser only contacts DNS when it needs to discover or refresh the server address.

Browser Cache and TTL

One important concept I learned is that browsers cache DNS responses.

For example:

Browser Cache

myportfolio.netlify.app

↓

104.xx.xx.xx

The browser continues using this cached IP until the TTL (Time To Live) expires.

When the TTL expires, the browser discards the cached IP address instead of assuming it is still correct.

It then asks the DNS Resolver for the latest IP address and stores the new result in its cache.

This allows hosting providers to change server IP addresses without browsers permanently using outdated information.

Why Use a CNAME Record?

Initially, I thought every domain needed its own IP address.

However, I learned that a **CNAME record does not store an IP address at all**.

Instead, it says:

"This domain is actually another domain."

For my portfolio, the DNS record will look like:

Field	Value
Host	VATSAL

Type CNAME

Value myportfolio.netlify.app

This means:

vatsal.flyrank.ai (will now be redirected to)



myportfolio.netlify.app

vatsal.flyrank.ai itself does not have an IP address.

Instead, DNS follows the CNAME and then finds the IP address of myportfolio.netlify.app.

Why CNAME Is Better Than an A Record

If I used an A Record, my DNS would directly store Netlify's IP address.

If Netlify later moved my website to another server, I would need to manually update that IP address.

Using a CNAME avoids this problem.

Suppose Netlify changes from:

104.xx.xx.10

to

104.xx.xx.55

Nothing changes in my DNS configuration.

When the browser's cached DNS record expires:

1. The browser discards the cached IP.
2. It asks the DNS Resolver again.
3. DNS follows the CNAME to myportfolio.netlify.app.
4. Netlify's DNS returns the **new** IP address.
5. The browser connects to the new server automatically.

Because of this, I never need to update my FlyRank DNS record when Netlify changes its infrastructure.

Verification Steps

After configuring the DNS record, I will verify that:

- <https://urvi.flyrank.ai> loads my portfolio.
- The original Netlify URL still works.
- HTTPS is enabled.
- All assets (CSS, JavaScript, and images) load correctly.
- The portfolio is accessible from both domains.